

Self-supervised FCNs for Monocular Depth-estimation

Oliver Tattersall (omtatter@uwaterloo.ca)

Abstract

This project aims to develop a model for monocular depth estimation using a self-supervised training approach, eliminating the need for ground truth disparity data. Self-supervised methods offer a cost-effective alternative to supervised learning, leveraging readily available stereo image pairs.

Inspired by the work of [\[Godard et al., 2017\]](#), we implement a fully convolutional network (FCN) architecture for disparity prediction. The model is trained on the KITTI stereo dataset, utilizing data augmentation techniques like random horizontal flips and color adjustments to improve generalization. Our approach incorporates a loss function combining appearance, disparity smoothness, left-right consistency, and disparity spread loss terms to guide the learning process.

Team Members & Contributions

For this project we largely worked together, and both of us contributed to all sections, however we started the project with these focuses:

- Oliver Tattersall (omtatter@uwaterloo.ca)
 - Reconstruction, Loss Functions, Training Loop
- Owen Waldron (orpwaldr@uwaterloo.ca)
 - Data Loading and Transforms, Model, Validation

Code Libraries

```
In [ ]: import random # Used in data augmentation
import copy

# Importing essential libraries for basic image manipulations.
import numpy as np
import PIL
from PIL import Image, ImageOps
import matplotlib.pyplot as plt

# We import some of the main PyTorch and TorchVision Libraries.
import torch
import torch.nn.functional as F
```

```
from torch import nn
from torch.utils.data import DataLoader, Subset, Dataset
import torchvision.transforms.functional as tF
from torchvision.datasets._stereo_matching import Kitti2015Stereo
```

Data Synthesis

We will be using a subset of the [KITTI stereo dataset](https://drive.google.com/file/d/1xltc0UvnsmSAEhGErjHEUHPkXvTcu3ZM/view?usp=sharing) for stereo disparity pairs. This subset is the KITTI 2015 stereo benchmark, and represents 200 training and testing stereo pairs taken from real world driving situations.

To download the dataset we used, we've uploaded a zip of our `datasets` folder at this google drive link:

<https://drive.google.com/file/d/1xltc0UvnsmSAEhGErjHEUHPkXvTcu3ZM/view?usp=sharing>

We take advantage of pytorch's builtin `Kitti2015Stereo` dataloader, however we are interested in exclusively the stereo image pairs, so we disregard the disparity map estimates and valid masks.

```
In [35]: # Wrap the Kitti2015Stereo dataloader to return the
class KittiStereo(Dataset):
    def __init__(self, *args, **kwargs):
        self.base_dataset = Kitti2015Stereo(*args, **kwargs)

    def __len__(self):
        return len(self.base_dataset)

    def __getitem__(self, idx):
        data = self.base_dataset[idx]
        return data[:2] # Only return the first two items (left & right images)
```

```
In [ ]: # Importing the KITTY dataset
# Make sure to have first downloaded the datasets folder from this link:
# https://drive.google.com/file/d/1xltc0UvnsmSAEhGErjHEUHPkXvTcu3ZM/view?usp=sharing
DATASET_PATH = 'datasets'

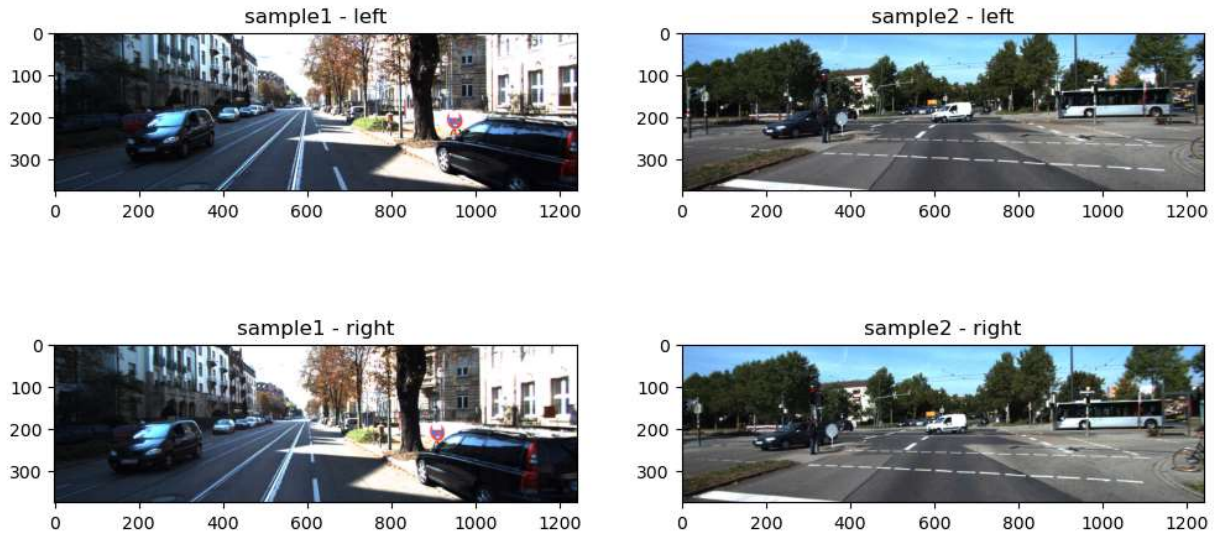
# Create a dataloader to load in images
sample1 = KittiStereo(DATASET_PATH)[0]
sample2 = KittiStereo(DATASET_PATH)[15]

left1, right1 = sample1
left2, right2 = sample2

fig = plt.figure(figsize=(12,6))
ax1 = fig.add_subplot(2,2,1)
plt.title('sample1 - left')
ax1.imshow(left1)
ax2 = fig.add_subplot(2,2,3)
plt.title('sample1 - right')
ax2.imshow(right1)
```

```
ax3 = fig.add_subplot(2,2,2)
plt.title('sample2 - left')
ax3.imshow(left2)
ax4 = fig.add_subplot(2,2,4)
plt.title('sample2 - right')
ax4.imshow(right2)
```

Out[]: <matplotlib.image.AxesImage at 0x7a0a20a4a030>



To augment training data, we first prepare the following transformations that follow the `StereoMatchingDataset` API. Note that that `StereoMatchingDataset` applies the transformations to the left & right images, as well as the disparities & valid_masks. These last two aren't used in training, so we do not bother transforming them in our transformations.

We use the same data augmentations as [Godard et al., 2017], including a random horizontal flip, as well as a random color augment. It is important that during the random flip, left and right images are swapped accordingly.

Also, the model architecture requires that the image dimensions be multiples of 128, so a `CropToMultiple` transformation was created to ensure this.

We also tried using a random resize crop similar to Assignment 5, but found that this resulted in much poorer results. This is likely due to the network needing context of the entire scene to estimate disparities, and disparity scale not being consistent with resizing.

```
In [37]: # Converts the PIL image to a tensor
class ToTensor():
    def __call__(self, images, disparities, valid_masks):
        images = (tf.to_tensor(images[0]), tf.to_tensor(images[1]))
        return images, disparities, valid_masks

# Flips images horizontally at random (& swaps Left and right)
class RandomFlip():
    def __init__(self, p=0.5):
        self.p = p
```

```

def __call__(self, images, disparities, valid_masks):
    if random.random() < self.p:
        return (tf.hflip(images[1]), tf.hflip(images[0])), disparities, valid_masks
    return images, disparities, valid_masks

# Randomly augment the images color by performing gamma, brightness, and
# rgb specific random
class RandomColorAugment():
    def __init__(self):
        self.rando = np.random.default_rng()

    def __call__(self, images, disparities, valid_masks):
        gamma = self.rando.uniform(0.8, 1.2)
        brightness = self.rando.uniform(0.5, 2.0)
        colors = self.rando.uniform(0.8, 1.2, size=3)

        def color_shift(img):
            arr = np.array(img).astype(np.float32) * colors # shape (H, W, 3)
            arr = np.clip(arr, 0, 255).astype(np.uint8)
            return Image.fromarray(arr)

        images = [tf.adjust_gamma(images[i], gamma) for i in range(2)]
        images = [tf.adjust_brightness(images[i], brightness) for i in range(2)]
        images = [color_shift(images[i]) for i in range(2)]

        return tuple(images), disparities, valid_masks

class CropToMultiple():
    def __init__(self, mult):
        self.mult = mult

    def center_crop_ndarray(self, arr, size):
        h, w = arr.shape[-2:]
        top = (h - size[0]) // 2
        left = (w - size[1]) // 2
        return arr[..., top:top+size[0], left:left+size[1]]

    def __call__(self, images, disparities, valid_masks):
        height = (images[0].height // self.mult) * self.mult
        width = (images[0].width // self.mult) * self.mult
        size = [height, width]

        images = (tf.center_crop(images[0], size), tf.center_crop(images[1], size))
        return images, disparities, valid_masks

class JointCompose():
    def __init__(self, transforms):
        self.transforms = transforms

    def __call__(self, images, disparities, valid_masks):
        for t in self.transforms:
            images, disparities, valid_masks = t(images, disparities, valid_masks)
        return images, disparities, valid_masks

```

```

In [38]: base = Kitti2015Stereo(
          DATASET_PATH,

```

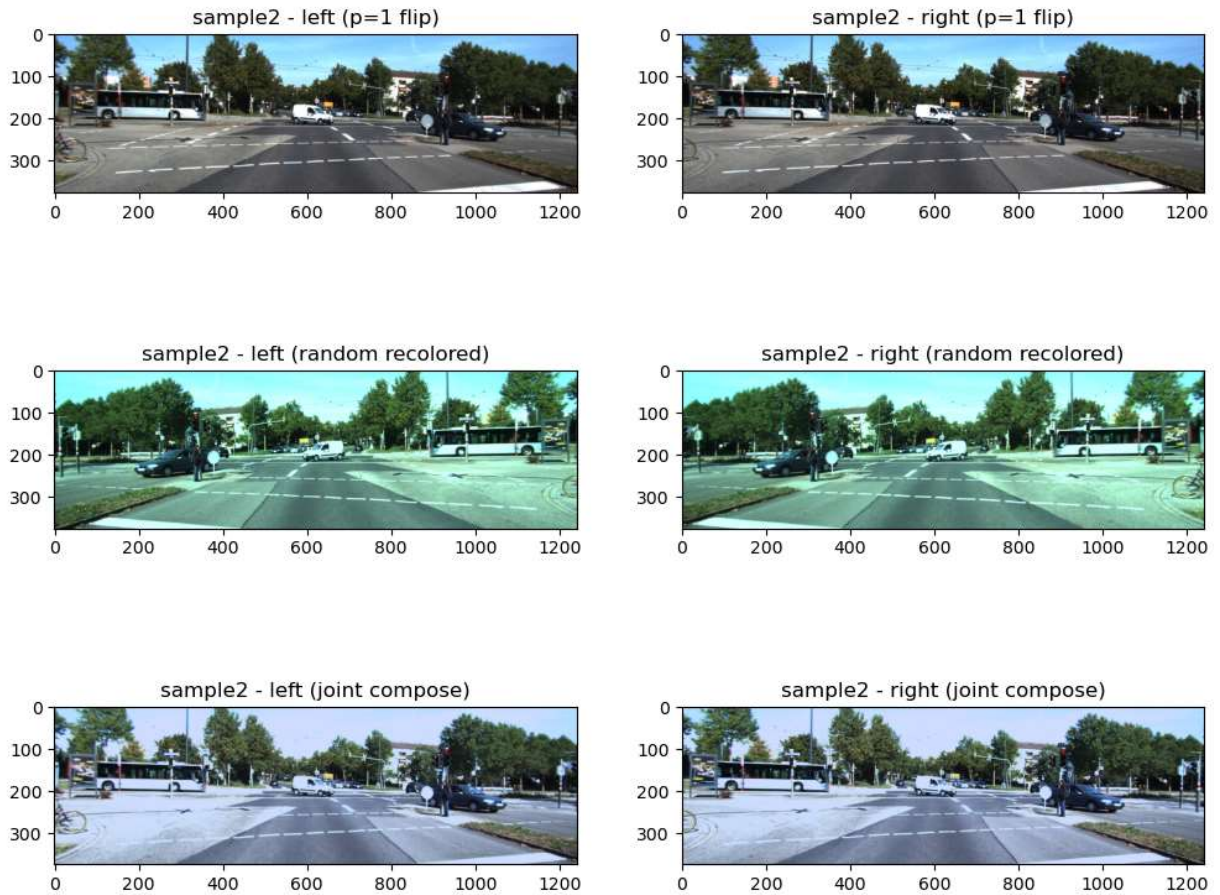
```

        split = 'train',
    )[15]
    flipped = Kitti2015Stereo(
        DATASET_PATH,
        split = 'train',
        transforms = JointCompose([RandomFlip(p=1)]),
    )[15]
    recolored = Kitti2015Stereo(
        DATASET_PATH,
        split = 'train',
        transforms = JointCompose([RandomColorAugment()]),
    )[15]
    joint = Kitti2015Stereo(
        DATASET_PATH,
        split = 'train',
        transforms = JointCompose([RandomFlip(p=1), RandomColorAugment()]),
    )[15]

    fig = plt.figure(figsize=(12,10))
    ax1 = fig.add_subplot(3,2,1)
    plt.title('sample2 - left (p=1 flip)')
    ax1.imshow(flipped[0])
    ax1 = fig.add_subplot(3,2,2)
    plt.title('sample2 - right (p=1 flip)')
    ax1.imshow(flipped[1])
    ax1 = fig.add_subplot(3,2,3)
    plt.title('sample2 - left (random recolored)')
    ax1.imshow(recolored[0])
    ax1 = fig.add_subplot(3,2,4)
    plt.title('sample2 - right (random recolored)')
    ax1.imshow(recolored[1])
    ax1 = fig.add_subplot(3,2,5)
    plt.title('sample2 - left (joint compose)')
    ax1.imshow(joint[0])
    ax1 = fig.add_subplot(3,2,6)
    plt.title('sample2 - right (joint compose)')
    ax1.imshow(joint[1])

```

Out[38]: <matplotlib.image.AxesImage at 0x7a0a2091cc80>



We will have 3 separate data loaders for different training contexts.

```
In [39]: sanity_data = Subset(KittiStereo(
    DATASET_PATH,
    split = 'train',
    transforms = JointCompose([CropToMultiple(128), ToTensor()]),
), indices=[6])

train_data = KittiStereo(
    DATASET_PATH,
    split = 'train',
    transforms = JointCompose([CropToMultiple(128), RandomColorAugment(), RandomFli
])

val_data = KittiStereo(
    DATASET_PATH,
    split='test',
    transforms = JointCompose([CropToMultiple(128), ToTensor()])
)

TRAIN_BATCH_SIZE = 4

sanity_loader = DataLoader(sanity_data, batch_size=1, shuffle=False)
train_loader = DataLoader(train_data, batch_size=TRAIN_BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_data, batch_size=1, shuffle=False)
```

Model

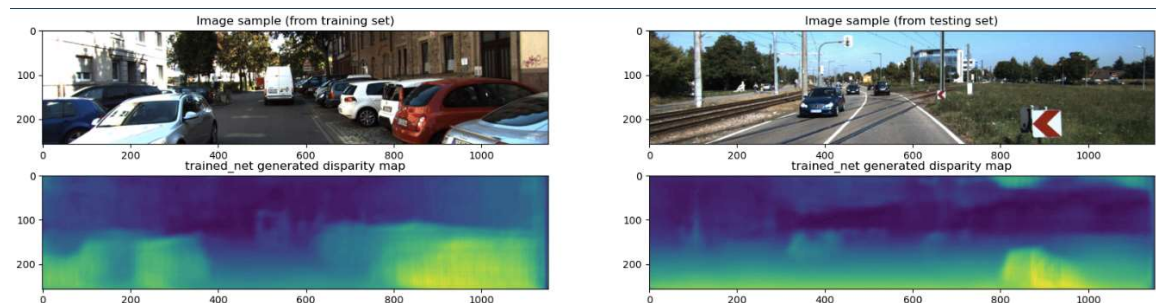
We follow the fully convolutional network architecture described by [Godard et al., 2017] with a few important differences. The model consists into an encoder and decoder. At a high level, the encoder recognizes features and the decoder reconstructs a disparity map given these identified features via skip connections from the encoder and upsampling.

The output of the network is **two** disparity maps for the left & right images respectively. In training, both are used to improve the output, but in evaluation only the left is used. The "disparities" are ranged [0, 1] and represent a scaled multiple of the image width.

The key differences we made to improve our results were:

- **Using a ReLU activation function as opposed to sigmoid for disparity outputs.** This was because we found sigmoid had the whole disparity map tend towards 0.5, which is not what we'd expect for the output.
- **Having the scale be a learnable parameter.** This allowed the model to learn how to scale its output appropriately.
- **Using exponentiated linear units (ELU) as an intermediate activation function.** After experimentation, we found this worked best for our model. This is likely because it encourages smoothness and centers around 0 since it allows negative values.

The decoder features several "upconvolutions" steps. We experimented using both transpose convolutions (where model learns to upsample) and a more simple bilinear interpolation followed by a convolution. We found transposed convolutions resulted in "checkerboard artifacts" as described by [Odena, et al., 2016]. You can observe this example of our trained output with transpose convolutions:



```
In [40]: ### model
class Model(nn.Module):
    def __init__(self, use_conv_transpose = False):
        super().__init__()

        # Scale parameter
        self.scale = nn.Parameter(torch.tensor(0.3))

        # Miscellaneous functions
        self.act_fn = nn.ELU(inplace=True)
        self.disp_act_fn = nn.ReLU()
        self.upsample2 = nn.Upsample(scale_factor=2, mode='bilinear')
```

```

# Encoder
self.conv1 = nn.Conv2d(3, 32, kernel_size=7, stride=2, padding=3)
self.conv1b = nn.Conv2d(32, 32, kernel_size=7, stride=1, padding=3)

self.conv2 = nn.Conv2d(32, 64, kernel_size=5, stride=2, padding=2)
self.conv2b = nn.Conv2d(64, 64, kernel_size=5, stride=1, padding=2)

self.conv3 = nn.Conv2d(64, 128, kernel_size=3, stride=2, padding=1)
self.conv3b = nn.Conv2d(128, 128, kernel_size=3, stride=1, padding=1)

self.conv4 = nn.Conv2d(128, 256, kernel_size=3, stride=2, padding=1)
self.conv4b = nn.Conv2d(256, 256, kernel_size=3, stride=1, padding=1)

self.conv5 = nn.Conv2d(256, 512, kernel_size=3, stride=2, padding=1)
self.conv5b = nn.Conv2d(512, 512, kernel_size=3, stride=1, padding=1)

self.conv6 = nn.Conv2d(512, 512, kernel_size=3, stride=2, padding=1)
self.conv6b = nn.Conv2d(512, 512, kernel_size=3, stride=1, padding=1)

self.conv7 = nn.Conv2d(512, 512, kernel_size=3, stride=2, padding=1)
self.conv7b = nn.Conv2d(512, 512, kernel_size=3, stride=1, padding=1)

# Decoder
self.upconv7 = nn.Sequential(
    nn.Upsample(scale_factor=2, mode='bilinear'),
    nn.Conv2d(512, 512, kernel_size=3, stride=1, padding=1)
)
self.iconv7 = nn.Conv2d(1024, 512, kernel_size=3, stride=1, padding=1)

self.upconv6 = nn.Sequential(
    nn.Upsample(scale_factor=2, mode='bilinear'),
    nn.Conv2d(512, 512, kernel_size=3, stride=1, padding=1)
)
self.iconv6 = nn.Conv2d(1024, 512, kernel_size=3, stride=1, padding=1)

self.upconv5 = nn.Sequential(
    nn.Upsample(scale_factor=2, mode='bilinear'),
    nn.Conv2d(512, 256, kernel_size=3, stride=1, padding=1)
)
self.iconv5 = nn.Conv2d(512, 256, kernel_size=3, stride=1, padding=1)

self.upconv4 = nn.Sequential(
    nn.Upsample(scale_factor=2, mode='bilinear'),
    nn.Conv2d(256, 128, kernel_size=3, stride=1, padding=1)
)
self.iconv4 = nn.Conv2d(256, 128, kernel_size=3, stride=1, padding=1)
self.disp4 = nn.Conv2d(128, 2, kernel_size=3, stride=1, padding=1)

self.upconv3 = nn.Sequential(
    nn.Upsample(scale_factor=2, mode='bilinear'),
    nn.Conv2d(128, 64, kernel_size=3, stride=1, padding=1)
)
self.iconv3 = nn.Conv2d(128 + 2, 64, kernel_size=3, stride=1, padding=1)
self.disp3 = nn.Conv2d(64, 2, kernel_size=3, stride=1, padding=1)

```

```

self.upconv2 = nn.Sequential(
    nn.Upsample(scale_factor=2, mode='bilinear'),
    nn.Conv2d(64, 32, kernel_size=3, stride=1, padding=1)
)
self.iconv2 = nn.Conv2d(64 + 2, 32, kernel_size=3, stride=1, padding=1)
self.disp2 = nn.Conv2d(32, 2, kernel_size=3, stride=1, padding=1)

self.upconv1 = nn.Sequential(
    nn.Upsample(scale_factor=2, mode='bilinear'),
    nn.Conv2d(32, 16, kernel_size=3, stride=1, padding=1)
)
self.iconv1 = nn.Conv2d(16 + 2, 16, kernel_size=3, stride=1, padding=1)
self.disp1 = nn.Conv2d(16, 2, kernel_size=3, stride=1, padding=1)

if use_conv_transpose:
    self.upconv7 = nn.ConvTranspose2d(512, 512, kernel_size=3, stride=2, padding=1)
    self.upconv6 = nn.ConvTranspose2d(512, 512, kernel_size=3, stride=2, padding=1)
    self.upconv5 = nn.ConvTranspose2d(512, 256, kernel_size=3, stride=2, padding=1)
    self.upconv4 = nn.ConvTranspose2d(256, 128, kernel_size=3, stride=2, padding=1)
    self.upconv3 = nn.ConvTranspose2d(128, 64, kernel_size=3, stride=2, padding=1)
    self.upconv2 = nn.ConvTranspose2d(64, 32, kernel_size=3, stride=2, padding=1)
    self.upconv1 = nn.ConvTranspose2d(32, 16, kernel_size=3, stride=2, padding=1)

def forward(self, left_image):
    # Encoder
    conv1 = self.act_fn(self.conv1(left_image))
    conv1b = self.act_fn(self.conv1b(conv1))
    conv2 = self.act_fn(self.conv2(conv1b))
    conv2b = self.act_fn(self.conv2b(conv2))
    conv3 = self.act_fn(self.conv3(conv2b))
    conv3b = self.act_fn(self.conv3b(conv3))
    conv4 = self.act_fn(self.conv4(conv3b))
    conv4b = self.act_fn(self.conv4b(conv4))
    conv5 = self.act_fn(self.conv5(conv4b))
    conv5b = self.act_fn(self.conv5b(conv5))
    conv6 = self.act_fn(self.conv6(conv5b))
    conv6b = self.act_fn(self.conv6b(conv6))
    conv7 = self.act_fn(self.conv7(conv6b))
    conv7b = self.act_fn(self.conv7b(conv7))

    # Decoder
    upconv7 = self.act_fn(self.upconv7(conv7b))
    iconv7 = self.act_fn(self.iconv7(torch.cat([upconv7, conv6b], dim=1)))
    upconv6 = self.act_fn(self.upconv6(iconv7))
    iconv6 = self.act_fn(self.iconv6(torch.cat([upconv6, conv5b], dim=1)))
    upconv5 = self.act_fn(self.upconv5(iconv6))
    iconv5 = self.act_fn(self.iconv5(torch.cat([upconv5, conv4b], dim=1)))
    upconv4 = self.act_fn(self.upconv4(iconv5))
    iconv4 = self.act_fn(self.iconv4(torch.cat([upconv4, conv3b], dim=1)))
    disp4 = self.scale * self.disp_act_fn(self.disp4(iconv4))
    upconv3 = self.act_fn(self.upconv3(iconv4))
    iconv3 = self.act_fn(self.iconv3(torch.cat([upconv3, conv2b, self.upsample
    disp3 = self.scale * self.disp_act_fn(self.disp3(iconv3))
    upconv2 = self.act_fn(self.upconv2(iconv3))
    iconv2 = self.act_fn(self.iconv2(torch.cat([upconv2, conv1b, self.upsample

```

```

disp2 = self.scale * self.disp_act_fn(self.disp2(iconv2))
upconv1 = self.act_fn(self.upconv1(iconv2))
iconv1 = self.act_fn(self.iconv1(torch.cat([upconv1, self.upsample2(disp2)])))
disp1 = self.scale * self.disp_act_fn(self.disp1(iconv1))

return disp1

```

Reconstructing the image from the disparity

To test the outputted disparities, part of the error is recreating the right image/left image and comparing to the actual right image/left image. To do that we created a function that shifts pixel location based on the disparity shift, and then normalizes and uses torch's bilinear grid sample to create a new image. The function assumes recreating a right image, so we have a wrapper that multiplies by -1 when recreating the left image instead.

```

In [41]: def bilinear_sampler_1d_h(img, disp_map):
    B, C, H, W = img.size()
    device = img.device

    if disp_map.dim() == 4 and disp_map.size(1) == 1:
        disp_map = disp_map.squeeze(1)

    # Create meshgrid in pixel coordinates.
    y_grid, x_grid = torch.meshgrid(
        torch.arange(H, device=device).float(),
        torch.arange(W, device=device).float(),
        indexing='ij'
    )
    x_grid = x_grid.unsqueeze(0).expand(B, H, W)
    y_grid = y_grid.unsqueeze(0).expand(B, H, W)

    # Scale the normalized disp_map into pixel units and update grid pixels
    x_offset_scaled = disp_map * W
    x_new = x_grid + x_offset_scaled

    # Now we have new pixel coordinates (x_new, y_grid).
    # Next, convert these pixel coordinates to normalized coordinates for grid_sample
    x_norm = 2 * x_new / (W - 1) - 1
    y_norm = 2 * y_grid / (H - 1) - 1

    # Construct the sampling grid. grid_sample expects a grid of shape [B, H, W, 2]
    grid = torch.stack((x_norm, y_norm), dim=3) # shape: [B, H, W, 2]

    # Use grid_sample to perform bilinear interpolation.
    output = F.grid_sample(img, grid, mode='bilinear', padding_mode='border', align_corners=True)

    return output

```

```

In [42]: ## LeftIm defines if we are reconstructing the left image or the right image
def reconStruct(im, disp, leftIm):
    if(leftIm):
        return bilinear_sampler_1d_h(im, -1*disp)

```

```

else:
    return bilinear_sampler_1d_h(im, disp)

```

Example on a ground truth image

```

In [43]: imL = Image.open("images/tsukuba/scene1.row3.col3.ppm")
gt = np.load("images/tsukuba/resMap.pgm").astype(np.float32)
gt = tF.to_tensor(gt)
gt = gt/gt.shape[2] #normalize into [0, 1]

tensImL = tF.to_tensor(imL)[None]

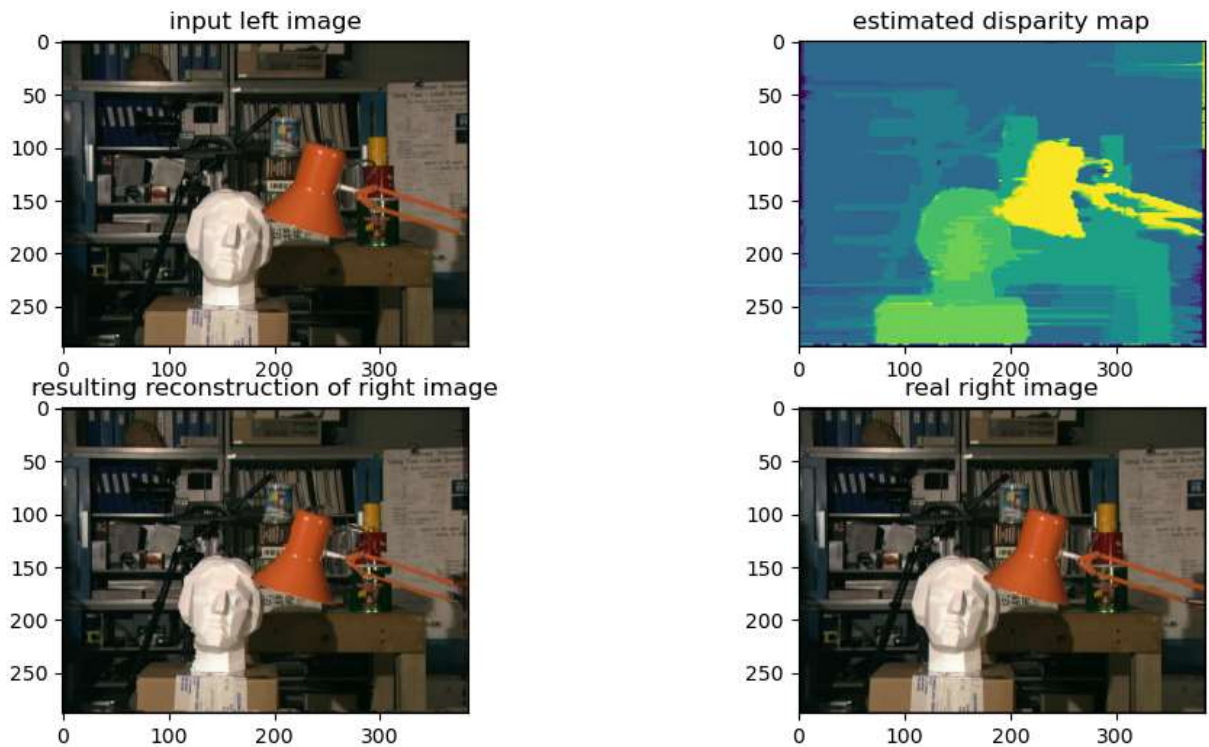
imR = Image.open("images/tsukuba/scene1.row3.col4.ppm")

regen_im = reconStruct(tensImL, gt[None], False)[0]

gt = gt.permute(1, 2, 0)
fig = plt.figure(figsize=(12,6))
ax1 = fig.add_subplot(2, 2, 1)
ax1.set_title("input left image")
ax1.imshow(imL)
ax2 = fig.add_subplot(2, 2, 2)
ax2.set_title("estimated disparity map")
ax2.imshow(gt)
ax3 = fig.add_subplot(2, 2, 3)
ax3.set_title("resulting reconstruction of right image")
ax3.imshow(regen_im.permute(1, 2, 0))
ax3 = fig.add_subplot(2, 2, 4)
ax3.set_title("real right image")
ax3.imshow(imR)

```

Out[43]: <matplotlib.image.AxesImage at 0x7a0a20635310>



```
In [44]: untrained_net = Model().eval()

untrained_net.cuda()

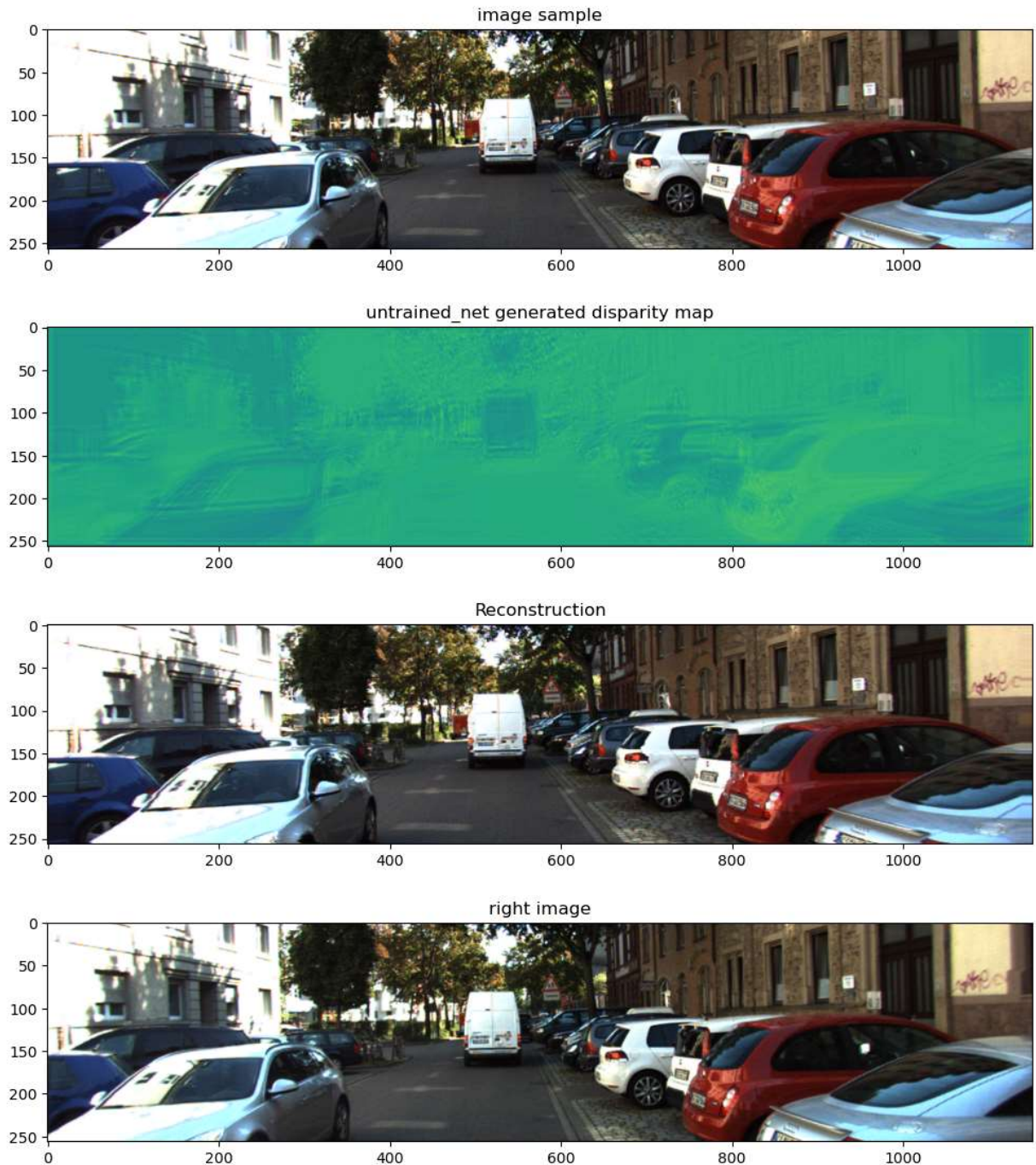
im1, imR = KittiStereo(
    DATASET_PATH,
    split = 'train',
    transforms = JointCompose([CropToMultiple(128), ToTensor()]),
)[6]

im1 = im1.cuda()

dL1 = untrained_net.forward(im1[None])[0, 0]
recon = reconStruct(im1[None].cpu().detach(), dL1[None].cpu().detach(), False)[0].p

fig = plt.figure(figsize=(12,14))
ax = fig.add_subplot(4,1,1)
plt.title('image sample')
ax.imshow(im1.cpu().permute(1, 2, 0).numpy())
ax = fig.add_subplot(4,1,2)
plt.title("untrained_net generated disparity map")
ax.imshow(dL1.cpu().detach().numpy())
ax = fig.add_subplot(4, 1, 3)
plt.title(f"Reconstruction")
ax.imshow(recon)
ax = fig.add_subplot(4, 1, 4)
plt.title("right image")
ax.imshow(imR.permute(1, 2, 0))

torch.save(untrained_net.state_dict(), "untrained_net.pth")
```



Loss Function

We took inspiration for our loss function from [\[Godard et al., 2017\]](#), which consisted of 3 parts: the appearance loss, the disparity smoothness loss, and the left-right consistency loss.

The appearance loss is the loss of the reconstructed images to their real counterparts. The paper initially used a mixture of SSIM(Structural Similarity Index Measure) and L1 norm, however we found that using a mean-square error resulted in much better results that more accurately generated the depth maps.

The disparity smoothness loss is a loss to reduce jumps in disparity, except possible at object boundaries. It does this by multiplying the gradient of the derivate by the negative exponential of the gradient of the image, for both x and y directions. The formula for this is $\frac{1}{N} \sum_{i,j} |\partial_x d_{ij}| e^{-\|\partial_x I_{ij}\|} + |\partial_y d_{ij}| e^{-\|\partial_y I_{ij}\|}$, which is applied to both the left and right image/disparity pairs generated by the model. In testing the model, we found that the disparities tended to be the same across the horizontal and simply decrease as it went upward, so we added a scaling factor to both the x and y terms, to increase the cost for disparity jumps in the y direction. This ended up encouraging better object prediction and separation, however the disparity line artifacts can still be found.

The left-right consistency loss is to ensure that the left and right disparity maps generated are closely related, by checking if the disparity on the shifted right disparity map are the same as the left disparity map. The formula for this function is $\frac{1}{N} \sum_{ij} |d_{ij}^l - d_{ij+d_{ij}^l}^r|$.

While this function allows for more smoothness than a simple photo-consistency term, an issue we ran into is that the disparity map being 0 results in a zero loss for both the disparity smoothness and left-right consistency losses. We tried to reduce the effect of these losses so the model didn't optimize for generating 0 disparity maps by decreasing the associated coefficient, however this ended up with very poor and spotty results. Instead, we added a loss term to encourage more uniformity and spread through the disparity_uniform_loss function, which is a negative exponential on the variance of the disparity. This allowed us to keep the other errors significant without running into the 0 disparity maps issue. None the less, the 0 disparity map was still a local minimum, so we would have to rerun the model whenever its first state was a 0 disparity map.

```
In [45]: def disparity_uniform_loss(values, lmbda = 10):
          return torch.exp(lmbda * -1 * values.var())

def gradient_x(img):
    return img[:, :, :, :-1] - img[:, :, :, 1:]

def gradient_y(img):
    return img[:, :, :-1, :] - img[:, :, 1:, :]

def disparity_smoothness(dis, img, x=1, y=2):
    disp_grad_x = gradient_x(dis)
    disp_grad_x[torch.isinf(disp_grad_x)] = 0
    disp_grad_x[torch.isnan(disp_grad_x)] = 0
    disp_grad_y = gradient_y(dis)
    disp_grad_y[torch.isinf(disp_grad_y)] = 0
    disp_grad_y[torch.isnan(disp_grad_y)] = 0
    img_grad_x = gradient_x(img)
    img_grad_y = gradient_y(img)

    weights_x = torch.exp(-torch.sum(torch.abs(img_grad_x), 1, keepdim=True))
    weights_y = torch.exp(-torch.sum(torch.abs(img_grad_y), 1, keepdim=True))

    smoothness_x = x*torch.abs(disp_grad_x * weights_x)
```

```

smoothness_y = y*disp_grad_y * weights_y
return smoothness_x.abs().mean() + smoothness_y.abs().mean()

def lr_consistency_loss(disp_left, disp_right):
    B, _, H, W = disp_left.shape

    # Create normalized horizontal coordinate grid
    grid_x = torch.linspace(-1.0, 1.0, W).view(1, 1, 1, W).expand(B, 1, H, W).to(device)
    grid_y = torch.linspace(-1.0, 1.0, H).view(1, 1, H, 1).expand(B, 1, H, W).to(device)

    disp_left_norm = disp_left
    disp_right_norm = disp_right

    # Sample right disparity using left disparity to warp
    grid_warped_right = torch.cat([grid_x - disp_left_norm, grid_y], dim=1)
    grid_warped_right = grid_warped_right.permute(0, 2, 3, 1)
    sampled_right = F.grid_sample(disp_right, grid_warped_right, mode='bilinear', padding_mode='zeros')

    # Sample left disparity using right disparity to warp
    grid_warped_left = torch.cat([grid_x + disp_right_norm, grid_y], dim=1)
    grid_warped_left = grid_warped_left.permute(0, 2, 3, 1)
    sampled_left = F.grid_sample(disp_left, grid_warped_left, mode='bilinear', padding_mode='zeros')

    # L1 loss between disparity and its warped counterpart
    loss_left = torch.abs(disp_left - sampled_right).mean()
    loss_right = torch.abs(disp_right - sampled_left).mean()

    return loss_left + loss_right

def appearance_loss(imL, imR, dL, dR, reconImL, reconImR):
    loss = nn.MSELoss()
    return loss(imR, reconImR) + loss(imL, reconImL)

def custom_loss(imL, imR, dL, dR, recImL, recImR, alpha_ap=5.0, alpha_ds=1e-4, alpha_lr=1.0, alpha_uni=1.0):
    loss_ap = appearance_loss(imL, imR, dL, dR, recImL, recImR)
    if debug: print("loss_ap", loss_ap)

    loss_ds = disparity_smoothness(dL, imR) + disparity_smoothness(dR, imL)
    if debug: print("loss_ds", loss_ds)

    loss_lr = lr_consistency_loss(dL, dR)
    if debug: print("loss_lr", loss_lr)

    loss_uni = disparity_uniform_loss(dL) + disparity_uniform_loss(dR)
    if debug: print("uni loss:", loss_uni)

    # --- Total Loss ---
    total_loss = alpha_ap * loss_ap + alpha_ds * loss_ds + alpha_lr * loss_lr + alpha_uni * loss_uni
    return total_loss

```

Testing the loss function

```
In [46]: imL =tf.to_tensor(Image.open("images/tsukuba/scene1.row3.col3.ppm"))[None]
imR =tf.to_tensor(Image.open("images/tsukuba/scene1.row3.col4.ppm"))[None]
dL = tf.to_tensor(Image.open("images/tsukuba/truedisp.row3.col3.pgm"))[None]
width = imL.shape[2]
dL /= width
dR = -1 * dL

zero = torch.zeros(dL.shape)

print("Good loss: ", custom_loss(imL, imR, dL, dR, reconStruct(imR, dR, True), reco
print("Bad loss: ", custom_loss(imL, imR, zero, zero, reconStruct(imR, zero, True),
print(disparity_uniform_loss(zero), disparity_uniform_loss(dL), torch.min(dL), torc
```

Good loss: tensor(0.2105)

Bad loss: tensor(0.2137)

tensor(1.) tensor(1.0000) tensor(0.) tensor(0.0031) tensor(0.0012) tensor(6.2768e-07)

Trainer

To train the model, we set up 2 simple, reusable functions that train by running each item getting the outputted left and right disparity maps, reconstructing the left and right image, and passing to loss function, from which we can store the loss value, and back propagate. Once it is done, we display the loss graph to ensure that there is downward motion in the loss graph and that the model is not just getting stuck.

For our optimizer, we initially used torch.optim.SGD, however it was resulting in slow optimization and grainy images. We switched to the same optimizer [\[Godard et al., 2017\]](#) used, which is the Adam optimizer

```
In [47]: def train(train_loader, net, optimizer, loss_graph, loss_function):

    net.train()
    # if USE_GPU:
    net = net.cuda()
    val_loss = 0
    for i, data in enumerate(train_loader):

        imL, imR = data
        # if USE_GPU:
        imL, imR = imL.cuda(), imR.cuda()

        optimizer.zero_grad()
        output = net.forward(imL)
        dL, dR = output[:, torch.newaxis, 0], output[:, torch.newaxis, 1]
        main_loss = loss_function(imL, imR, dL, dR, reconStruct(imR, dR, True), reco
        loss = main_loss.item()
        val_loss += loss
        loss_graph.append(loss)
        main_loss.backward()
        optimizer.step()
```

```
# Return the value of loss (averaged over the loaded data)
return val_loss/len(train_loader)
```

```
In [48]: def train_net(loader, net, optimizer, EPOCH, loss_function = custom_loss):
net.train()

print("Starting Training...")

loss_graph = []

fig = plt.figure(figsize=(12,6))
plt.subplots_adjust(bottom=0.2,right=0.85,top=0.95)
ax = fig.add_subplot(1,1,1)

for e in range(EPOCH):
    loss = train(loader, net, optimizer, loss_graph, loss_function)
    ax.clear()
    ax.set_xlabel('iterations')
    ax.set_ylabel('loss value')
    ax.set_title('Training loss curve for OVERFIT_NET')
    ax.plot(loss_graph, label='training loss')
    ax.legend(loc='upper right')
    fig.canvas.draw()
    print("Epoch: {} Loss: {}".format(e, loss))
```

```
In [49]: def get_optimizer(net):
optimizer = torch.optim.Adam(net.parameters(), lr=1e-4, betas=(0.9, 0.999), eps
return optimizer
```

Overfit model

First, we are going to train an overfit net to a single image to get a sanity check that the error function and model are working as intended

```
In [50]: %%time

overfit_net = copy.deepcopy(untrained_net)

EPOCH = 500

optimizer = get_optimizer(overfit_net)

train_net(sanity_loader, overfit_net, optimizer, EPOCH)

torch.save(overfit_net.state_dict(), "overfit_net.pth")
```

Starting Training...

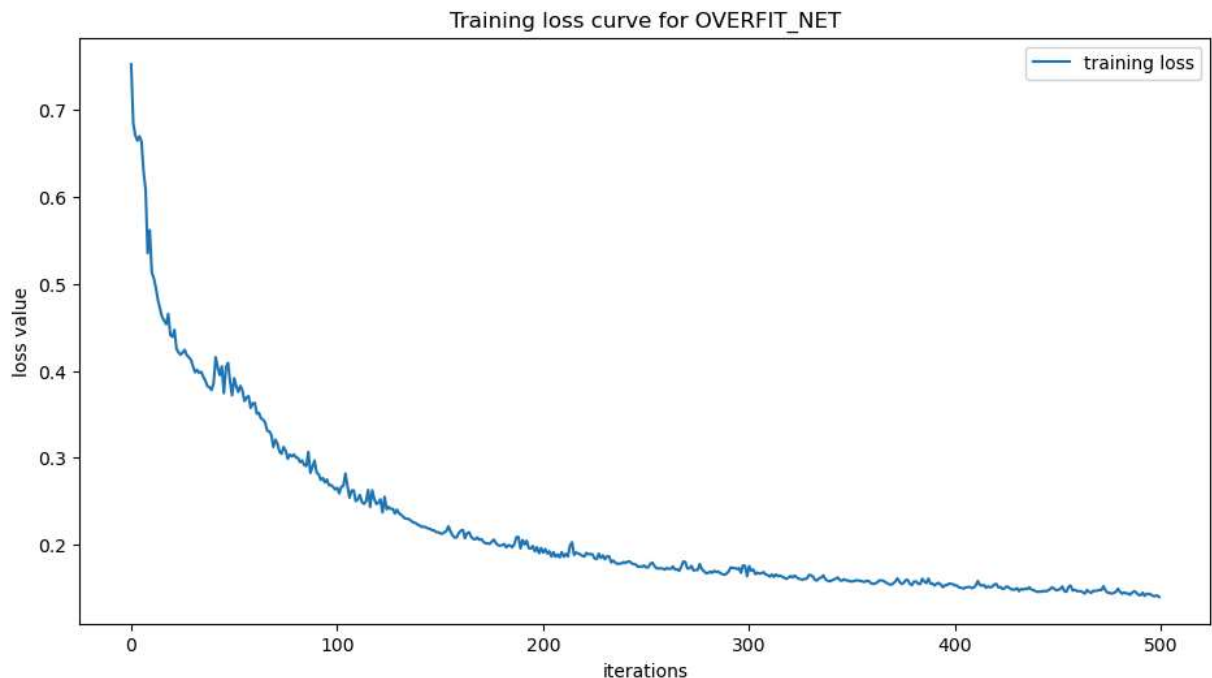
Epoch: 0 Loss: 0.7523955702781677
Epoch: 1 Loss: 0.6845204830169678
Epoch: 2 Loss: 0.6708881855010986
Epoch: 3 Loss: 0.664618730545044
Epoch: 4 Loss: 0.6696719527244568
Epoch: 5 Loss: 0.6634878516197205
Epoch: 6 Loss: 0.6294675469398499
Epoch: 7 Loss: 0.6082991361618042
Epoch: 8 Loss: 0.5352538824081421
Epoch: 9 Loss: 0.5618956089019775
Epoch: 10 Loss: 0.5132452249526978
Epoch: 11 Loss: 0.50616854429245
Epoch: 12 Loss: 0.49529320001602173
Epoch: 13 Loss: 0.4812227487564087
Epoch: 14 Loss: 0.47168633341789246
Epoch: 15 Loss: 0.4617261290550232
Epoch: 16 Loss: 0.457678884267807
Epoch: 17 Loss: 0.45356088876724243
Epoch: 18 Loss: 0.46546444296836853
Epoch: 19 Loss: 0.44112610816955566

...

Epoch: 490 Loss: 0.14197461307048798
Epoch: 491 Loss: 0.1448998898267746
Epoch: 492 Loss: 0.14128264784812927
Epoch: 493 Loss: 0.14373604953289032
Epoch: 494 Loss: 0.14351613819599152
Epoch: 495 Loss: 0.14280058443546295
Epoch: 496 Loss: 0.14123481512069702
Epoch: 497 Loss: 0.14088958501815796
Epoch: 498 Loss: 0.14176684617996216
Epoch: 499 Loss: 0.1399751752614975

CPU times: user 3min 17s, sys: 15.5 s, total: 3min 32s

Wall time: 35.7 s



Now we run it on its own image and a random image in the set to compare its results

```

In [51]: overfit_net.eval()

im1, imR = KittiStereo(
    DATASET_PATH,
    split = 'train',
    transforms = JointCompose([CropToMultiple(128), ToTensor()]),
)[6]

im1 = im1.cuda()

dL1 = overfit_net.forward(im1[None])[0, 0]

fig = plt.figure(figsize=(20,10))
ax = fig.add_subplot(4,2,1)
plt.title('Image sample (from training set)')
ax.imshow(im1.cpu().permute(1, 2, 0).numpy())
ax = fig.add_subplot(4,2,3)
plt.title("overfit_net generated disparity map")
ax.imshow(dL1.cpu().detach().numpy())
ax = fig.add_subplot(4,2,5)
plt.title("Reconstructed right image")
ax.imshow(reconStruct(im1[None].cpu().detach(), dL1[None].cpu().detach(), False)[0])
ax = fig.add_subplot(4,2,7)
plt.title("Ground truth right image")
ax.imshow(imR.permute(1,2,0))

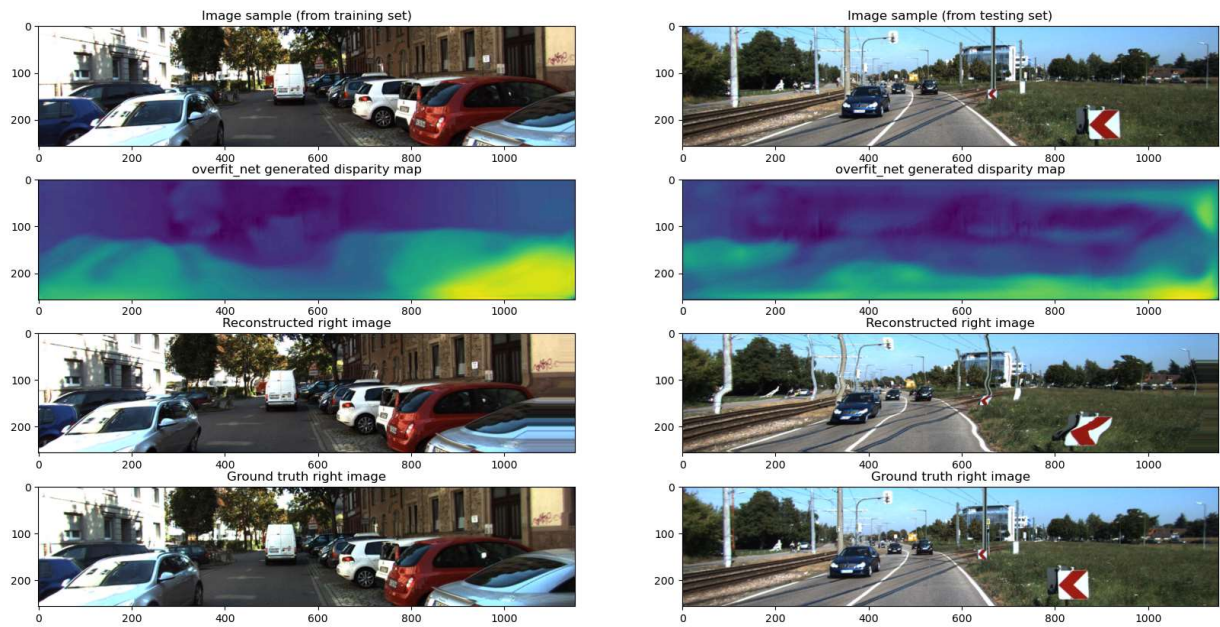
im1, imR = KittiStereo(
    DATASET_PATH,
    split = 'test',
    transforms = JointCompose([CropToMultiple(128), ToTensor()]),
)[15]

im1 = im1.cuda()
dL1 = overfit_net.forward(im1[None])[0, 0]

ax = fig.add_subplot(4,2,2)
plt.title('Image sample (from testing set)')
ax.imshow(im1.cpu().permute(1, 2, 0).numpy())
ax = fig.add_subplot(4,2,4)
plt.title("overfit_net generated disparity map")
ax.imshow(dL1.cpu().detach().numpy())
ax = fig.add_subplot(4,2,6)
plt.title("Reconstructed right image")
ax.imshow(reconStruct(im1[None].cpu().detach(), dL1[None].cpu().detach(), False)[0])
ax = fig.add_subplot(4,2,8)
plt.title("Ground truth right image")
ax.imshow(imR.permute(1, 2, 0))

```

```
Out[51]: <matplotlib.image.AxesImage at 0x7a0a1bf43800>
```



Trained Model

```
In [52]: %%time

trained_net = copy.deepcopy(untrained_net)

EPOCH = 200

optimizer = get_optimizer(trained_net)

# train_net(train_loader, trained_net, optimizer, EPOCH, test_loss)
train_net(train_loader, trained_net, optimizer, EPOCH)

torch.save(trained_net.state_dict(), "trained_net.pth")
```

Starting Training...

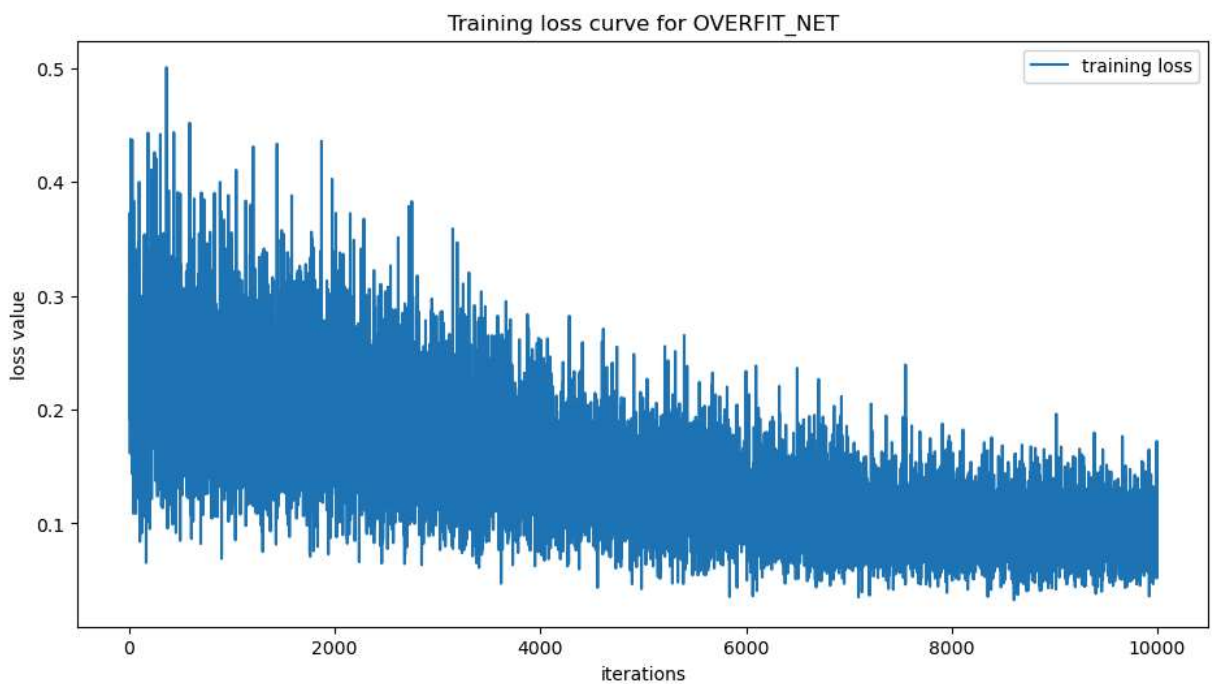
Epoch: 0 Loss: 0.2691592951118946
Epoch: 1 Loss: 0.22936688989400864
Epoch: 2 Loss: 0.2101681910455227
Epoch: 3 Loss: 0.22383389621973038
Epoch: 4 Loss: 0.2233053669333458
Epoch: 5 Loss: 0.229795892983675
Epoch: 6 Loss: 0.2261324493587017
Epoch: 7 Loss: 0.21173692777752876
Epoch: 8 Loss: 0.21026850774884223
Epoch: 9 Loss: 0.21687042742967605
Epoch: 10 Loss: 0.21592384755611418
Epoch: 11 Loss: 0.2168129290640354
Epoch: 12 Loss: 0.2008751904964447
Epoch: 13 Loss: 0.22292229041457176
Epoch: 14 Loss: 0.2148070240020752
Epoch: 15 Loss: 0.21760707408189772
Epoch: 16 Loss: 0.208220424503088
Epoch: 17 Loss: 0.21552711918950082
Epoch: 18 Loss: 0.22237197533249856
Epoch: 19 Loss: 0.20726191774010658
Epoch: 20 Loss: 0.2151028834283352

...

Epoch: 190 Loss: 0.09213640384376048
Epoch: 191 Loss: 0.09180591806769371
Epoch: 192 Loss: 0.0858558701723814
Epoch: 193 Loss: 0.0909931156039238
Epoch: 194 Loss: 0.08478554517030716
Epoch: 195 Loss: 0.09012155339121819
Epoch: 196 Loss: 0.08535087116062641
Epoch: 197 Loss: 0.09000770784914494
Epoch: 198 Loss: 0.08650413386523724
Epoch: 199 Loss: 0.08667033091187477

CPU times: user 2h 48min 14s, sys: 13min 22s, total: 3h 1min 36s

Wall time: 30min 16s



Similarly, we compare visually the results compared to the overfit image and another random image

```
In [53]: # ckpt = torch.load("decentmodels/firstNoTransform/trained_net.pth", weights_only=True)
# trained_net.load_state_dict(ckpt)
trained_net.eval()

im1, imR = KittiStereo(
    DATASET_PATH,
    split = 'train',
    transforms = JointCompose([CropToMultiple(128), ToTensor()]),
)[6]

im1 = im1.cuda()

dL1 = trained_net.forward(im1[None])[0, 0]

# print(torch.min(dL1).item(), torch.max(dL1).item(), torch.mean(dL1).item(), torch
# print(disparity_uniform_loss(dL1))

fig = plt.figure(figsize=(20,10))
ax = fig.add_subplot(4,2,1)
plt.title('Image sample (from training set)')
ax.imshow(im1.cpu().permute(1, 2, 0).numpy())
ax = fig.add_subplot(4,2,3)
plt.title("trained_net generated disparity map")
ax.imshow(dL1.cpu().detach().numpy())
ax = fig.add_subplot(4,2,5)
plt.title("Reconstructed right image")
ax.imshow(reconStruct(im1[None].cpu().detach(), dL1[None].cpu().detach(), False)[0])
ax = fig.add_subplot(4,2,7)
plt.title("Ground truth right image")
ax.imshow(imR.permute(1,2,0))

im1, imR = KittiStereo(
    DATASET_PATH,
    split = 'test',
    transforms = JointCompose([CropToMultiple(128), ToTensor()]),
)[15]

im1 = im1.cuda()
dL1 = trained_net.forward(im1[None])[0, 0]

# print(torch.min(dL1).item(), torch.max(dL1).item(), torch.mean(dL1).item(), torch
# print("disparity_uniform_loss(dL1).item())

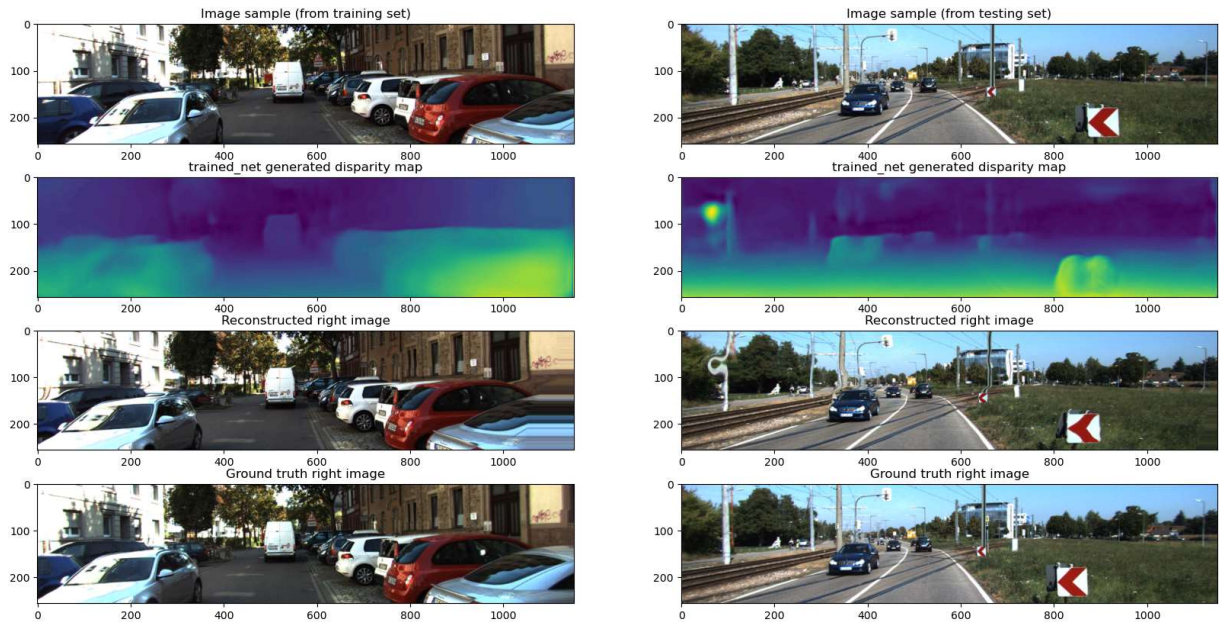
ax = fig.add_subplot(4,2,2)
plt.title('Image sample (from testing set)')
ax.imshow(im1.cpu().permute(1, 2, 0).numpy())
ax = fig.add_subplot(4,2,4)
plt.title("trained_net generated disparity map")
```

```

ax.imshow(dL1.cpu().detach().numpy())
ax = fig.add_subplot(4,2,6)
plt.title("Reconstructed right image")
ax.imshow(reconStruct(im1[None].cpu().detach(), dL1[None].cpu().detach(), False)[0])
ax = fig.add_subplot(4,2,8)
plt.title("Ground truth right image")
ax.imshow(imR.permute(1, 2, 0))

```

Out[53]: <matplotlib.image.AxesImage at 0x7a0a201edf10>



We can also see how the model performs with images far different from the driving landscape scenes it was trained on by seeing how it fares with the Tsukuba example from class:

```

In [54]: gt = tf.to_tensor(Image.open("images/tsukuba/truedisp.row3.col3.pgm"))

imL, imR = JointCompose([CropToMultiple(128), ToTensor()])(Image.open("images/tsukuba/truedisp.row3.col3.pgm"))
imL, imR = imL.cuda(), imR.cuda()

dL = trained_net.forward(imL[None])[0, 0]

regen_im = reconStruct(imL[None].cpu().detach(), dL[None].cpu().detach(), False)[0]

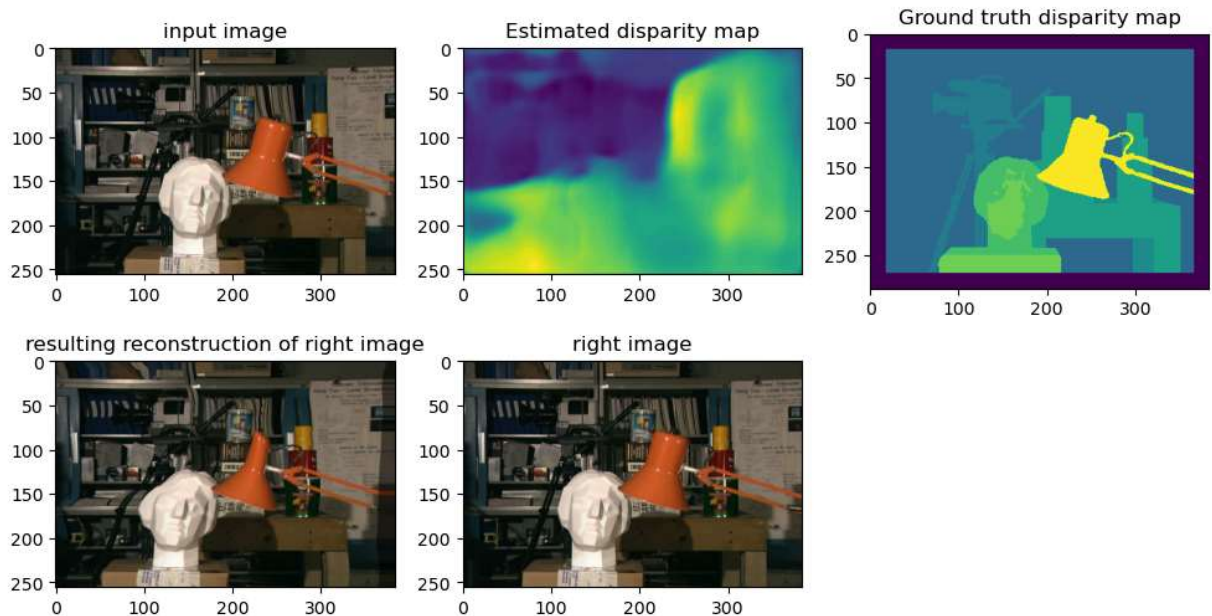
fig = plt.figure(figsize=(12,6))
ax1 = fig.add_subplot(2, 3, 1)
ax1.set_title("input image")
ax1.imshow(imL.cpu().detach().permute(1, 2, 0))
ax2 = fig.add_subplot(2, 3, 2)
ax2.imshow(dL.cpu().detach())
ax2.set_title("Estimated disparity map")
ax2 = fig.add_subplot(2, 3, 3)
ax2.imshow(gt.permute(1, 2, 0))
ax2.set_title("Ground truth disparity map")
ax3 = fig.add_subplot(2, 3, 4)
ax3.set_title("resulting reconstruction of right image")

```

```
ax3.imshow(regen_im)

ax3 = fig.add_subplot(2, 3, 5)
ax3.set_title("right image")
ax3.imshow(imR.cpu().detach().permute(1, 2, 0))
```

Out[54]: <matplotlib.image.AxesImage at 0x7a0a203c0c80>



Quantitative Validation

Without using ground truth disparities, it is difficult to construct meaningful validation metric. We started with the structural similarity index measure (SSIM), used as an intermediate in [Godard et al., 2017]'s loss function. This resulted in values of around 0.995, 0.996, and 0.998 for untrained, overfit, and trained models over the validation dataset. These high values are due to the fact that stereo image pairs already have much structure in common.

If we instead use a simple photometric loss (being the mean absolute difference across all pixels between the expected and reproduced right images), we get slightly more meaningful results. The photometric losses decrease from 0.111 and 0.101 for untrained and overfit to 0.066 for the validation dataset after training. This shows that the overtrained model performs only slightly better, but doesn't learn to generalize to unseen driving scenes as well as the trained model does.

```
In [55]: def validate(val_loader, net):
          net.eval()
          val_loss = 0

          net = net.cuda()
          with torch.no_grad():
              for i, data in enumerate(val_loader):
```

```

imL, imR = data
imL, imR = imL.cuda(), imR.cuda()

dL = net.forward(imL)[0, 0]

recon_imR = reconStruct(imL, dL[None], False)[0].permute(1, 2, 0)

photometric_loss = torch.abs(imR - recon_imR.permute(2, 0, 1)[None]).me

val_loss += photometric_loss

return val_loss/len(val_loader.dataset)

```

```

In [56]: %%time
print("Photometric loss for UNTRAINED_NET over the validation dataset:{}".format(va
print("Photometric loss for OVERFIT_NET over the training dataset:{}".format(valida
print("Photometric loss for OVERFIT_NET over the validation dataset:{}".format(vali
print("Photometric loss for TRAINED_NET over the training dataset:{}".format(valida
print("Photometric loss for TRAINED_NET over the validation dataset:{}".format(vali

```

```

Photometric loss for UNTRAINED_NET over the validation dataset:0.0939188078045845
Photometric loss for OVERFIT_NET over the training dataset:0.06902937591075897
Photometric loss for OVERFIT_NET over the validation dataset:0.1048368439078331
Photometric loss for TRAINED_NET over the training dataset:0.06387476623058319
Photometric loss for TRAINED_NET over the validation dataset:0.06719004362821579
CPU times: user 2min 22s, sys: 11.4 s, total: 2min 34s
Wall time: 25.7 s

```

Conclusion

In the final analysis, we were mostly successful in creating a FCN capable of monocular depth estimation. Although it is not without error, the results from the trained model show the capacity to generalize beyond the training data and make meaningful estimates in a scene it was not trained on. We can observe this qualitatively in general accuracy of the depth maps produced, as well as quantitatively where the trained model performed as well on the unseen validation dataset as the training dataset. One limitation is the inability to generalize beyond the driving/landscape scenes it is familiar with, as observed in its failure on the Tsuksuba example from class.

Although the results are far from being as substantial as that of [\[Godard et al., 2017\]](#), our model trained on a much smaller subset of the data in much less time. Whereas they used took "25 hours" to train on "30 thousand images", we only trained on a set of 200 images in around 30 minutes. The simplicities and adjustments we made allowed our model to be trained more efficiently at the cost of accuracy. Perhaps in this right it is better suited for less critical, more specialized situations than automated driving.

As a continuation, we might explore how the model performs when trained on a different dataset than driving situations. Another exploration could involve assessing

efficiency/accuracy tradeoff when using more than just the final disparity layer in training ([Godard et al., 2017] uses all 4 `disp` layers, for example).

Sources

Godard, Clément, et al. "Unsupervised Monocular Depth Estimation with Left-Right Consistency." Computer Vision and Pattern Recognition, 21 July 2017, <https://doi.org/10.1109/cvpr.2017.699>.

Odena, Augustus, et al. "Deconvolution and Checkerboard Artifacts." Distill, vol. 1, no. 10, 17 Oct. 2016, <https://doi.org/10.23915/distill.00003>.